

Đồ thị cơ bản (BFS/DFS)

Nội dung

1. Đồ thị là gì? Ví dụ thực tế
2. Biểu diễn đồ thị bằng danh sách kề (Adjacency List)
 - Code đọc đồ thị
3. DFS (Depth-First Search) — Tìm kiếm theo chiều sâu
 - Ý tưởng
 - Thuật toán
 - Truy vết trực quan
 - Code DFS
 - Ứng dụng quan trọng: Đếm thành phần liên thông
4. BFS (Breadth-First Search) — Tìm kiếm theo chiều rộng
 - Ý tưởng
 - Thuật toán
 - Truy vết trực quan
 - Tại sao BFS tìm được đường đi ngắn nhất?
 - Code BFS
5. Flood Fill trên lưới — Đếm phòng (Counting Rooms)
 - Cách giải
6. BFS tìm đường đi ngắn nhất + Truy vết đường đi
 - Cách giải
7. Đồ thị hai phía (Bipartite Graph) — Tô 2 màu
 - Đồ thị hai phía là gì?
 - Cách kiểm tra: BFS/DFS tô màu
 - Code kiểm tra Bipartite bằng BFS
8. Topological Sort — Sắp xếp topo (Đồ thị có hướng không chu trình)
 - Bài toán thực tế
 - Thuật toán Kahn (BFS-based)
 - Truy vết
 - Code
9. Sai lầm phổ biến khi làm bài đồ thị

10. Mẹo luyện tập

Chào mừng các bạn đến với đồ thị (Graph) — một trong những cấu trúc dữ liệu quan trọng nhất trong khoa học máy tính và lập trình thi đấu! Rất nhiều bài toán thực tế có thể mô hình hóa bằng đồ thị, và BFS/DFS là hai thuật toán nền tảng nhất để xử lý chúng.

1. Đồ thị là gì? Ví dụ thực tế

Đồ thị gồm hai thành phần:

- Đỉnh (vertex/node): đại diện cho các đối tượng.
- Cạnh (edge): đại diện cho mối quan hệ giữa hai đối tượng.

Ví dụ thực tế:

BÀI TOÁN	ĐỈNH	CẠNH
Bản đồ giao thông	Thành phố	Đường nối 2 thành phố
Mạng xã hội	Người dùng	Quan hệ bạn bè
Internet	Máy tính	Kết nối mạng
Lịch học	Môn học	Môn A phải học trước môn B
Mê cung	Ô trong lưới	2 ô liền kề đi được

Phân loại:

- Đồ thị vô hướng (undirected): cạnh (u, v) đi được cả hai chiều. Ví dụ: đường hai chiều, quan hệ bạn bè.
- Đồ thị có hướng (directed): cạnh (u, v) chỉ đi được từ u đến v . Ví dụ: đường một chiều, môn tiên quyết.

Hãy xem một đồ thị vô hướng đơn giản với 5 đỉnh và 6 cạnh: Các cạnh: $(1, 2)$, $(1, 3)$, $(2, 3)$, $(2, 4)$, $(3, 4)$,

2. Biểu diễn đồ thị bằng danh sách kề (Adjacency List)

Có nhiều cách lưu đồ thị trong máy tính. Cách phổ biến nhất trong CP là danh sách kề (adjacency list).

Ý tưởng: Với mỗi đỉnh u , lưu danh sách các đỉnh v kề với u (tức các đỉnh v sao cho có cạnh (u, v)).

Ví dụ với đồ thị trên:

```
adj[1] = {2, 3}
adj[2] = {1, 3, 4}
adj[3] = {1, 2, 4}
adj[4] = {2, 3, 5}
adj[5] = {4}
```

Code đọc đồ thị

Input mẫu (5 đỉnh, 6 cạnh):

```
5 6
1 2
1 3
2 3
2 4
3 4
4 5
```

C++

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n, m; // n đỉnh, m cạnh
    cin >> n >> m;

    // Tạo danh sách kề (đánh số đỉnh từ 1 đến n)
    vector<vector<int>> adj(n + 1);

    for (int i = 0; i < m; i++) {
        int u, v;
        cin >> u >> v;
        adj[u].push_back(v); // thêm v vào danh sách kề của u
        adj[v].push_back(u); // thêm u vào danh sách kề của v (đồ thị vô hướng)
        // Nếu đồ thị CÓ HƯỚNG: chỉ giữ dòng adj[u].push_back(v)
    }
}
```

Python

```
n, m = map(int, input().split())

# Tạo danh sách kề, đánh số đỉnh từ 1 đến n
adj = [[] for _ in range(n + 1)]

for _ in range(m):
    u, v = map(int, input().split())
    adj[u].append(v) # thêm v vào danh sách kề của u
    adj[v].append(u) # thêm u vào danh sách kề của v (đồ thị vô hướng)
    # Nếu đồ thị CÓ HƯỚNG: chỉ giữ dòng adj[u].append(v)
```

Bộ nhớ: $O(n + m)$ — rất tiết kiệm!

Lưu ý: Còn cách biểu diễn bằng ma trận kề ($O(n^2)$ bộ nhớ), nhưng chỉ phù hợp khi $n \leq 1000$. Trong CP, danh sách kề là lựa chọn tiêu chuẩn.

3. DFS (Depth-First Search) — Tìm kiếm theo chiều sâu

Ý tưởng

DFS đi sâu nhất có thể trước khi quay lui. Giống như bạn đi vào mê cung: luôn rẽ theo một hướng cho đến khi đụng tường, rồi quay lại và thử hướng khác.

Thuật toán

1. Bắt đầu từ đỉnh u .
2. Đánh dấu u là "đã thăm".
3. Với mỗi đỉnh v kề u :
4. Nếu v chưa thăm $\rightarrow$ đệ quy DFS(v).
5. Khi không còn đỉnh kề chưa thăm $\rightarrow$ quay lui.

Truy vết trực quan

DFS bắt đầu từ đỉnh 1:

- **Bước 1:** Thăm 1 (đánh dấu `visited[1] = true`). Hàng xóm: [2, 3]. Chọn 2 (chưa thăm) $\rightarrow$ đệ quy DFS(2).
- **Bước 2:** Thăm 2 (đánh dấu `visited[2] = true`). Hàng xóm: [1, 3, 4]. 1 đã thăm $\rightarrow$ bỏ qua. Chọn 3 $\rightarrow$ đệ quy DFS(3).
- **Bước 3:** Thăm 3 (đánh dấu `visited[3] = true`). Hàng xóm: [1, 2, 4]. 1 đã thăm, 2 đã thăm. Chọn 4 $\rightarrow$ đệ quy DFS(4).
- **Bước 4:** Thăm 4 (đánh dấu `visited[4] = true`). Hàng xóm: [2, 3, 5]. 2 đã thăm, 3 đã thăm. Chọn 5 $\rightarrow$ đệ quy DFS(5).
- **Bước 5:** Thăm 5 (đánh dấu `visited[5] = true`). Hàng xóm: [4]. 4 đã thăm $\rightarrow$ không đi đâu được $\rightarrow$ Quay lui về 4.
- Quay lui về 4: Không còn hàng xóm chưa thăm $\rightarrow$ quay lui về 3.
- Quay lui về 3: Không còn $\rightarrow$ quay lui về 2.
- Quay lui về 2: Không còn $\rightarrow$ quay lui về 1.
- Quay lui về 1: Không còn $\rightarrow$ KẾT THÚC.

Thứ tự thăm: $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5$.

Ngăn xếp đệ quy (recursion stack) tại bước 5:

```
| DFS(5) | <- đỉnh stack
| DFS(4) |
| DFS(3) |
| DFS(2) |
| DFS(1) | <- đáy stack
```

Code DFS

C++

```
#include <bits/stdc++.h>
using namespace std;

const int MAXN = 100005;
vector<int> adj[MAXN];
bool visited[MAXN];

void dfs(int u) {
    visited[u] = true; // Đánh dấu đã thăm
    cout << u << " "; // In ra thứ tự thăm (tùy bài)
    for (int v : adj[u]) { // Duyệt tất cả hàng xóm
        if (!visited[v]) { // Nếu chưa thăm
            dfs(v);        // đệ quy DFS
        }
    }
}

int main() {
    int n, m;
    cin >> n >> m;
    for (int i = 0; i < m; i++) {
        int u, v;
        cin >> u >> v;
        adj[u].push_back(v);
        adj[v].push_back(u);
    }
    memset(visited, false, sizeof visited);
    dfs(1); // Bắt đầu DFS từ đỉnh 1
}
```

Python

```
import sys
sys.setrecursionlimit(10**6)

n, m = map(int, input().split())
adj = [[] for _ in range(n + 1)]

for _ in range(m):
    u, v = map(int, input().split())
    adj[u].append(v)
    adj[v].append(u)

visited = [False] * (n + 1)

def dfs(u):
```

```

visited[u] = True # Đánh dấu đã thăm
print(u, end=' ') # In ra thứ tự thăm (tùy bài)
for v in adj[u]: # Duyệt tất cả hàng xóm
    if not visited[v]: # Nếu chưa thăm
        dfs(v)        # đệ quy DFS

dfs(1) # Bắt đầu DFS từ đỉnh 1

```

Độ phức tạp: $O(n + m)$ — mỗi đỉnh và mỗi cạnh được xét đúng 1 lần.

Ứng dụng quan trọng: Đếm thành phần liên thông

Thành phần liên thông (connected component) là nhóm các đỉnh mà từ bất kỳ đỉnh nào trong nhóm đều có thể đi đến các đỉnh khác.

Ví dụ đồ thị với 7 đỉnh có 3 thành phần liên thông: $\{1, 2, 3\}$, $\{4, 5, 6\}$, $\{7\}$.

Cách đếm: Lặp qua tất cả đỉnh, nếu đỉnh chưa thăm $\rightarrow$ gọi DFS $\rightarrow$ tăng bộ đếm.

C++

```

int components = 0;
for (int u = 1; u <= n; u++) {
    if (!visited[u]) {
        dfs(u);
        components++;
    }
}
cout << components << endl;

```

Python

```

components = 0
for u in range(1, n + 1):
    if not visited[u]:
        dfs(u)
        components += 1
print(components)

```

4. BFS (Breadth-First Search) — Tìm kiếm theo chiều rộng

Ý tưởng

BFS duyệt từng lớp (level): thăm tất cả đỉnh cách gốc 1 bước trước, rồi tất cả đỉnh cách gốc 2 bước, rồi 3 bước... Giống như sóng lan tỏa từ tâm: sóng lan đến đâu thì thăm đến đó.

BFS dùng hàng đợi (queue) — cấu trúc FIFO (vào trước, ra trước).

Thuật toán

1. Đưa đỉnh nguồn vào queue, đánh dấu khoảng cách = 0.
2. Lặp cho đến khi queue rỗng:
 - a. Lấy đỉnh u ra khỏi đầu queue.
 - b. Với mỗi đỉnh kề v chưa thăm của u :
 - Khoảng cách $v = \text{khoảng cách } u + 1$.
 - Đưa v vào cuối queue.

Truy vết trực quan

Cùng đồ thị, BFS từ đỉnh 1:

- **Bước 0:** Queue = [1], dist = [-, 0, -, -, -, -]. Lấy 1 ra. Hàng xóm chưa thăm: 2, 3. Thêm 2 và 3 vào queue $\rightarrow$ Queue = [2, 3], dist = [-, 0, 1, 1, -, -].
- **Bước 1:** Lấy 2 ra. Hàng xóm: 1 (đã thăm), 3 (đã thăm), 4 (chưa). Thêm 4 vào queue $\rightarrow$ Queue = [3, 4], dist = [-, 0, 1, 1, 2, -].
- **Bước 2:** Lấy 3 ra. Hàng xóm: 1 (đã thăm), 2 (đã thăm), 4 (đã thăm). Không thêm gì $\rightarrow$ Queue = [4], dist = [-, 0, 1, 1, 2, -].
- **Bước 3:** Lấy 4 ra. Hàng xóm: 2 (đã thăm), 3 (đã thăm), 5 (chưa). Thêm 5 vào queue $\rightarrow$ Queue = [5], dist = [-, 0, 1, 1, 2, 3].
- **Bước 4:** Lấy 5 ra. Hàng xóm: 4 (đã thăm). Không thêm gì $\rightarrow$ Queue = [] $\rightarrow$ KẾT THÚC.

Kết quả khoảng cách (số cạnh ngắn nhất từ đỉnh 1):

Đỉnh:	1	2	3	4	5
dist:	0	1	1	2	3

Tại sao BFS tìm được đường đi ngắn nhất?

Vì BFS duyệt theo từng lớp: tất cả đỉnh cách nguồn k bước được thăm trước tất cả đỉnh cách nguồn $k + 1$ bước. Nên lần đầu tiên ta đến một đỉnh, đó chắc chắn là đường ngắn nhất!

Lưu ý quan trọng: BFS chỉ tìm đường ngắn nhất khi mỗi cạnh có trọng số bằng nhau (hoặc không trọng số). Nếu cạnh có trọng số khác nhau, phải dùng thuật toán Dijkstra.

Code BFS

C++

```
#include <bits/stdc++.h>
using namespace std;

const int MAXN = 100005;
vector<int> adj[MAXN];
```

```

int dist[MAXN]; // dist[v] = khoảng cách ngắn nhất từ nguồn đến v

void bfs(int start, int n) {
    memset(dist, -1, sizeof dist); // -1 = chưa thăm
    queue<int> q;

    dist[start] = 0;
    q.push(start);

    while (!q.empty()) {
        int u = q.front();
        q.pop();

        for (int v : adj[u]) {
            if (dist[v] == -1) { // v chưa thăm?
                dist[v] = dist[u] + 1; // Khoảng cách = khoảng cách u + 1
                q.push(v); // Đưa v vào queue
            }
        }
    }
}

int main() {
    int n, m;
    cin >> n >> m;
    for (int i = 0; i < m; i++) {
        int u, v;
        cin >> u >> v;
        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    bfs(1, n); // BFS từ đỉnh 1

    // In khoảng cách từ đỉnh 1 đến tất cả đỉnh khác
    for (int i = 1; i <= n; i++) {
        cout << "Đỉnh " << i << ": " << dist[i] << endl;
    }
}

```

Python

```

from collections import deque

n, m = map(int, input().split())
adj = [[] for _ in range(n + 1)]

for _ in range(m):
    u, v = map(int, input().split())

```

```

adj[u].append(v)
adj[v].append(u)

dist = [-1] * (n + 1) # dist[v] = khoảng cách ngắn nhất từ nguồn đến v

def bfs(start):
    q = deque()
    dist[start] = 0
    q.append(start)

    while q:
        u = q.popleft()
        for v in adj[u]:
            if dist[v] == -1: # v chưa thăm?
                dist[v] = dist[u] + 1 # khoảng cách = khoảng cách u + 1
                q.append(v) # đưa v vào queue

bfs(1) # BFS từ đỉnh 1

for i in range(1, n + 1):
    print(f"Đỉnh {i}: {dist[i]}")

```

5. Flood Fill trên lưới — Đếm phòng (Counting Rooms)

Flood fill là ứng dụng kinh điển của DFS/BFS trên lưới 2D.

Đề bài: Cho bản đồ $n \times m$ gồm '.' (sàn) và '#' (tường). Đếm số phòng (một phòng là một vùng sàn liên thông theo 4 hướng).

Ví dụ:

```

#####
#..#...#
####.#.#
#..#...#
#####

```

Có 3 phòng:

```

#####
#AA#BBB#
####B#B#
#CC#BBB#
#####

```

Phòng A: 2 ô. Phòng B: 7 ô. Phòng C: 2 ô.

Cách giải

Biến lưới thành đồ thị: mỗi ô '.' là một đỉnh, hai ô '.' kề nhau (lên/xuống/trái/phải) có cạnh nối. Đếm thành phần liên thông = đếm phòng.

C++

```
#include <bits/stdc++.h>
using namespace std;

int n, m;
char grid[1005][1005];
bool visited[1005][1005];

// 4 hướng: lên, xuống, trái, phải
int dx[] = {-1, 1, 0, 0};
int dy[] = {0, 0, -1, 1};

void dfs(int x, int y) {
    visited[x][y] = true;
    for (int d = 0; d < 4; d++) {
        int nx = x + dx[d];
        int ny = y + dy[d];
        // Kiểm tra: trong lưới, là sàn, chưa thăm
        if (nx >= 0 && nx < n && ny >= 0 && ny < m
            && grid[nx][ny] == '.' && !visited[nx][ny]) {
            dfs(nx, ny);
        }
    }
}

int main() {
    cin >> n >> m;
    for (int i = 0; i < n; i++)
        cin >> grid[i];

    memset(visited, false, sizeof visited);
    int rooms = 0;

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            if (grid[i][j] == '.' && !visited[i][j]) {
                dfs(i, j); // Flood fill: đánh dấu toàn bộ phòng
                rooms++;   // Tìm thấy 1 phòng mới
            }
        }
    }
    cout << rooms << endl;
}
```

Python

```
import sys
sys.setrecursionlimit(10**6)
```

```

n, m = map(int, input().split())
grid = [list(input().strip()) for _ in range(n)]
visited = [[False] * m for _ in range(n)]

# 4 hướng: lên, xuống, trái, phải
dx = [-1, 1, 0, 0]
dy = [0, 0, -1, 1]

def dfs(x, y):
    visited[x][y] = True
    for d in range(4):
        nx = x + dx[d]
        ny = y + dy[d]
        # Kiểm tra: trong lưới, là sàn, chưa thăm
        if 0 <= nx < n and 0 <= ny < m and grid[nx][ny] == '.' and not visited[nx][ny]:
            dfs(nx, ny)

rooms = 0
for i in range(n):
    for j in range(m):
        if grid[i][j] == '.' and not visited[i][j]:
            dfs(i, j) # Flood fill: đánh dấu toàn bộ phòng
            rooms += 1 # Tìm thấy 1 phòng mới

print(rooms)

```

Mẹo: Thay vì dùng mảng `visited`, bạn có thể đổi ô `'.'` thành `'#'` sau khi thăm (tiết kiệm bộ nhớ):

C++

```

void dfs(int x, int y) {
    grid[x][y] = '#'; // Đánh dấu bằng cách "lấp" ô
    for (int d = 0; d < 4; d++) {
        int nx = x + dx[d], ny = y + dy[d];
        if (nx >= 0 && nx < n && ny >= 0 && ny < m && grid[nx][ny] == '.')
            dfs(nx, ny);
    }
}

```

Python

```

def dfs(x, y):
    grid[x][y] = '#' # Đánh dấu bằng cách "lấp" ô
    for d in range(4):
        nx = x + dx[d]
        ny = y + dy[d]

```

```
if 0 <= nx < n and 0 <= ny < m and grid[nx][ny] == '.':  
    dfs(nx, ny)
```

6. BFS tìm đường đi ngắn nhất + Truy vết đường đi

Đề bài: Cho mê cung $n \times m$. Tìm đường đi ngắn nhất từ ô 'A' đến ô 'B'. In ra đường đi dưới dạng chuỗi L/R/U/D.

Ví dụ:

```
#####  
#.A...#  
#####  
#.....#  
###.B#  
#####
```

Cách giải

BFS từ ô A. Ngoài mảng `dist`, ta lưu thêm mảng `parent` để biết "ô này đến từ ô nào", từ đó truy vết đường đi.

C++

```
#include <bits/stdc++.h>  
using namespace std;  
  
int main() {  
    int n, m;  
    cin >> n >> m;  
    vector<string> grid(n);  
    pair<int,int> A, B;  
  
    for (int i = 0; i < n; i++) {  
        cin >> grid[i];  
        for (int j = 0; j < m; j++) {  
            if (grid[i][j] == 'A') A = {i, j};  
            if (grid[i][j] == 'B') B = {i, j};  
        }  
    }  
  
    vector<vector<int>> dist(n, vector<int>(m, -1));  
    vector<vector<pair<int,int>>> parent(n, vector<pair<int,int>>(m, {-1, -1}));  
  
    int dx[] = {-1, 1, 0, 0};  
    int dy[] = {0, 0, -1, 1};  
    char dc[] = {'U', 'D', 'L', 'R'};  
  
    queue<pair<int,int>> q;
```

```

q.push(A);
dist[A.first][A.second] = 0;

while (!q.empty()) {
    auto [r, c] = q.front(); q.pop();
    for (int d = 0; d < 4; d++) {
        int nr = r + dx[d], nc = c + dy[d];
        if (0 <= nr && nr < n && 0 <= nc && nc < m
            && grid[nr][nc] != '#' && dist[nr][nc] == -1) {
            dist[nr][nc] = dist[r][c] + 1;
            parent[nr][nc] = {r, c};
            q.push({nr, nc});
        }
    }
}

auto [er, ec] = B;
if (dist[er][ec] == -1) {
    cout << "NO\n";
    return 0;
}

cout << "YES\n" << dist[er][ec] << '\n';

string path;
for (auto [r, c] = B; make_pair(r, c) != A; ) {
    auto [pr, pc] = parent[r][c];
    for (int d = 0; d < 4; d++) {
        if (pr + dx[d] == r && pc + dy[d] == c) {
            path += dc[d];
            break;
        }
    }
    r = pr; c = pc;
}
reverse(path.begin(), path.end());
cout << path << '\n';
}

```

Python

```

from collections import deque

n, m = map(int, input().split())
grid = [input().strip() for _ in range(n)]

dist = [[-1] * m for _ in range(n)]
parent = [(-1, -1) for _ in range(m)] for _ in range(n)]

```

```

dx = [-1, 1, 0, 0]
dy = [0, 0, -1, 1]
dc = ['U', 'D', 'L', 'R']

for i in range(n):
    for j in range(m):
        if grid[i][j] == 'A':
            A = (i, j)
        if grid[i][j] == 'B':
            B = (i, j)

q = deque([A])
dist[A[0]][A[1]] = 0

while q:
    r, c = q.popleft()
    for d in range(4):
        nr, nc = r + dx[d], c + dy[d]
        if 0 <= nr < n and 0 <= nc < m and grid[nr][nc] != '#' and dist[nr][nc] == -1:
            dist[nr][nc] = dist[r][c] + 1
            parent[nr][nc] = (r, c)
            q.append((nr, nc))

er, ec = B
if dist[er][ec] == -1:
    print("NO")
else:
    print("YES")
    print(dist[er][ec])
    path = []
    r, c = B
    while (r, c) != A:
        pr, pc = parent[r][c]
        for d in range(4):
            if (pr + dx[d], pc + dy[d]) == (r, c):
                path.append(dc[d])
                break
        r, c = pr, pc
    print(''.join(reversed(path)))

```

Truy vết hoạt động như thế nào? Giả sử BFS tìm được đường $A \rightarrow (2,3) \rightarrow (3,3) \rightarrow (3,4) \rightarrow B$. Ta lưu:

$$\begin{aligned}
 \text{parent}[B] &= (3,4) \\
 \text{parent}[(3,4)] &= (3,3) \\
 \text{parent}[(3,3)] &= (2,3) \\
 \text{parent}[(2,3)] &= A
 \end{aligned}$$

Truy ngược: $B \rightarrow (3,4) \rightarrow (3,3) \rightarrow (2,3) \rightarrow A$, rồi đảo ngược thành $A \rightarrow (2,3) \rightarrow (3,3) \rightarrow (3,4) \rightarrow B$.

7. Đồ thị hai phía (Bipartite Graph) — Tô 2 màu

Đồ thị hai phía là gì?

Đồ thị hai phía (bipartite graph) là đồ thị mà ta có thể tô 2 màu cho các đỉnh sao cho không có cạnh nào nối 2 đỉnh cùng màu.

Ví dụ thực tế: Chia học sinh thành 2 đội sao cho không có ai trong cùng đội là bạn thân.

Ví dụ đồ thị hai phía:

- Đỏ: 1, 3
- Xanh: 2, 4, 5
- Cạnh: (1,2), (1,4), (3,2), (3,4), (4,5) → Không cạnh nào nối 2 đỉnh cùng màu ✓

Ví dụ KHÔNG phải đồ thị hai phía: tam giác 1-2-3 (cạnh 1-2, 2-3, 3-1). Dù tô thế nào, luôn có 2 đỉnh kề cùng màu.

Cách kiểm tra: BFS/DFS tô màu

Thuật toán:

1. Bắt đầu từ một đỉnh, tô màu 0.
2. Tất cả hàng xóm tô màu 1.
3. Tất cả hàng xóm của hàng xóm tô màu 0.
4. Nếu gặp đỉnh đã tô mà cùng màu với đỉnh hiện tại → KHÔNG phải đồ thị hai phía.

Truy vết ví dụ — Đồ thị: 1-2, 2-3, 3-4, 4-1, 2-4

- Bước 1: Tô đỉnh 1 màu 0.
 - Bước 2: Hàng xóm 1 = {2, 4}. Tô 2 màu 1, tô 4 màu 1.
 - Bước 3: Hàng xóm 2 = {1, 3, 4}.
 - 1 đã tô màu 0, khác màu 2 (màu 1) → OK ✓
 - Tô 3 màu 0.
 - 4 đã tô màu 1, cùng màu 2 (màu 1) → XUNG ĐỘT! ✗
- Không phải đồ thị hai phía.

Code kiểm tra Bipartite bằng BFS

C++

```
#include <bits/stdc++.h>
using namespace std;

const int MAXN = 100005;
vector<int> adj[MAXN];
int color[MAXN]; // -1 = chưa tô, 0 hoặc 1
```

```

bool bfs_bipartite(int start) {
    queue<int> q;
    color[start] = 0;
    q.push(start);

    while (!q.empty()) {
        int u = q.front();
        q.pop();

        for (int v : adj[u]) {
            if (color[v] == -1) {
                // Chưa tô -> tô màu khác u
                color[v] = 1 - color[u];
                q.push(v);
            } else if (color[v] == color[u]) {
                // Đã tô cùng màu -> không phải bipartite!
                return false;
            }
        }
    }
    return true;
}

int main() {
    int n, m;
    cin >> n >> m;
    for (int i = 0; i < m; i++) {
        int u, v;
        cin >> u >> v;
        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    memset(color, -1, sizeof color);
    bool is_bipartite = true;

    // Kiểm tra từng thành phần liên thông
    for (int u = 1; u <= n; u++) {
        if (color[u] == -1) {
            if (!bfs_bipartite(u)) {
                is_bipartite = false;
                break;
            }
        }
    }

    cout << (is_bipartite ? "YES" : "NO") << endl;
}

```

Python

```
from collections import deque

n, m = map(int, input().split())
adj = [[] for _ in range(n + 1)]

for _ in range(m):
    u, v = map(int, input().split())
    adj[u].append(v)
    adj[v].append(u)

color = [-1] * (n + 1) # -1 = chưa tô, 0 hoặc 1

def bfs_bipartite(start):
    q = deque()
    color[start] = 0
    q.append(start)

    while q:
        u = q.popleft()
        for v in adj[u]:
            if color[v] == -1:
                # Chưa tô -> tô màu khác u
                color[v] = 1 - color[u]
                q.append(v)
            elif color[v] == color[u]:
                # Đã tô cùng màu -> không phải bipartite
                return False
    return True

is_bipartite = True

# Kiểm tra từng thành phần liên thông
for u in range(1, n + 1):
    if color[u] == -1:
        if not bfs_bipartite(u):
            is_bipartite = False
            break

print("YES" if is_bipartite else "NO")
```

Tính chất quan trọng: Đồ thị hai phía nếu và chỉ nếu không chứa chu trình có độ dài lẻ.

8. Topological Sort — Sắp xếp topo (Đồ thị có hướng không chu trình)

Bài toán thực tế

Lịch học: Bạn có n môn học, một số môn phải học trước môn khác. Tìm thứ tự học sao cho mọi điều kiện tiên quyết đều được thỏa mãn.

Ví dụ 6 môn học:

- Toán rời rạc (1) $\rightarrow$ Cấu trúc dữ liệu (3)
- Lập trình C (2) $\rightarrow$ Cấu trúc dữ liệu (3)
- Cấu trúc dữ liệu (3) $\rightarrow$ Thuật toán (5)
- Cấu trúc dữ liệu (3) $\rightarrow$ Cơ sở dữ liệu (4)
- Thuật toán (5) $\rightarrow$ AI (6)
- Cơ sở dữ liệu (4) $\rightarrow$ AI (6)

Đồ thị có hướng:

```
1 -> 3 -> 5 -> 6
2      4
```

Thứ tự hợp lệ: 1, 2, 3, 4, 5, 6 hoặc 2, 1, 3, 5, 4, 6 (nhiều đáp án đúng).

Thuật toán Kahn (BFS-based)

Ý tưởng: Dùng bậc vào (in-degree) — số cạnh đi vào mỗi đỉnh.

1. Tính bậc vào (in-degree) cho tất cả đỉnh.
2. Đưa tất cả đỉnh có bậc vào = 0 vào queue (đây là những đỉnh không có tiên quyết).
3. Lặp:
4. Lấy đỉnh u ra khỏi queue $\rightarrow$ thêm vào kết quả.
5. Với mỗi đỉnh v mà u chỉ đến: giảm $\text{in-degree}[v]$ đi 1.
6. Nếu $\text{in-degree}[v] = 0 \rightarrow$ đưa v vào queue.
7. Nếu kết quả có đủ n đỉnh $\rightarrow$ topo sort thành công. Nếu không $\rightarrow$ có chu trình!

Truy vết

Ban đầu: $\text{in-degree} = [-, 0, 0, 2, 1, 1, 2]$ (Đỉnh 1: 0, Đỉnh 2: 0, Đỉnh 3: 2, ...)

- **Bước 1:** Queue = [1, 2] ($\text{in-degree} = 0$). Lấy 1. Con: 3. $\text{in-degree}[3] = 2 - 1 = 1$.
Kết quả: [1].
- **Bước 2:** Queue = [2]. Lấy 2. Con: 3. $\text{in-degree}[3] = 1 - 1 = 0 \rightarrow$ thêm 3 vào queue.
Kết quả: [1, 2].

- **Bước 3:** Queue = [3]. Lấy 3. Con: 5, 4. $\text{in-degree}[5] = 0 \rightarrow$ thêm. $\text{in-degree}[4] = 0 \rightarrow$ thêm. Kết quả: [1, 2, 3].
 - **Bước 4:** Queue = [5, 4]. Lấy 5. Con: 6. $\text{in-degree}[6] = 2 - 1 = 1$. Kết quả: [1, 2, 3, 5].
 - **Bước 5:** Queue = [4]. Lấy 4. Con: 6. $\text{in-degree}[6] = 1 - 1 = 0 \rightarrow$ thêm. Kết quả: [1, 2, 3, 5, 4].
 - **Bước 6:** Queue = [6]. Lấy 6. Không có con. Kết quả: [1, 2, 3, 5, 4, 6].
- Thứ tự topo: 1, 2, 3, 5, 4, 6 ✓

Code

C++

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n, m;
    cin >> n >> m;
    vector<vector<int>> adj(n + 1);
    vector<int> in_degree(n + 1, 0);

    for (int i = 0; i < m; i++) {
        int u, v;
        cin >> u >> v;
        adj[u].push_back(v);
        in_degree[v]++;
    }

    // Đưa đỉnh có in-degree = 0 vào queue
    queue<int> q;
    for (int i = 1; i <= n; i++) {
        if (in_degree[i] == 0) {
            q.push(i);
        }
    }

    vector<int> topo_order;
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        topo_order.push_back(u);

        for (int v : adj[u]) {
            in_degree[v]--;
            if (in_degree[v] == 0) {
                q.push(v);
            }
        }
    }
}
```

```

    }

    if ((int)topo_order.size() != n) {
        cout << "IMPOSSIBLE" << endl; // Có chu trình!
    } else {
        for (int x : topo_order) {
            cout << x << " ";
        }
        cout << endl;
    }
}
}

```

Python

```

from collections import deque

n, m = map(int, input().split())
adj = [[] for _ in range(n + 1)]
in_degree = [0] * (n + 1)

for _ in range(m):
    u, v = map(int, input().split())
    adj[u].append(v)
    in_degree[v] += 1

# Đưa đỉnh có in-degree = 0 vào queue
q = deque()
for i in range(1, n + 1):
    if in_degree[i] == 0:
        q.append(i)

topo_order = []
while q:
    u = q.popleft()
    topo_order.append(u)

    for v in adj[u]:
        in_degree[v] -= 1
        if in_degree[v] == 0:
            q.append(v)

if len(topo_order) != n:
    print("IMPOSSIBLE") # Có chu trình
else:
    print(*topo_order)

```

9. Sai lầm phổ biến khi làm bài đồ thị

Sai lầm 1: Quên kiểm tra visited

C++

```
// SAI - sẽ lặp vô hạn hoặc thăm đỉnh nhiều lần!
void dfs(int u) {
    for (int v : adj[u])
        dfs(v);
}

// ĐÚNG
void dfs(int u) {
    visited[u] = true;
    for (int v : adj[u])
        if (!visited[v])
            dfs(v);
}
```

Python

```
# SAI - sẽ lặp vô hạn hoặc thăm đỉnh nhiều lần!
def dfs(u):
    for v in adj[u]:
        dfs(v)

# ĐÚNG
def dfs(u):
    visited[u] = True
    for v in adj[u]:
        if not visited[v]:
            dfs(v)
```

Sai lầm 2: Tràn stack (Stack Overflow) với DFS đệ quy

Khi đồ thị là đường thẳng (1-2-3-...-100000), DFS đệ quy sẽ gọi 100000 lần lồng nhau → tràn stack! Tuy nhiên điều này chỉ diễn ra trên máy tính cá nhân, còn các online judge đều có stack lớn nên không lo bị tràn.

Giải pháp: Dùng BFS (luôn an toàn), hoặc DFS dùng stack thủ công:

C++

```
void dfs_iterative(int start) {
    stack<int> st;
    st.push(start);
    visited[start] = true;
    while (!st.empty()) {
        int u = st.top();
        st.pop();
        for (int v : adj[u]) {
            if (!visited[v]) {
```

```

        visited[v] = true;
        st.push(v);
    }
}
}

```

Python

```

def dfs_iterative(start):
    st = [start]
    visited[start] = True
    while st:
        u = st.pop()
        for v in adj[u]:
            if not visited[v]:
                visited[v] = True
                st.append(v)

```

Hoặc tăng stack/giới hạn chạy tùy môi trường:

- Linux/macOS: chạy chương trình với stack lớn hơn, ví dụ: `ulimit -s unlimited` rồi `./a.out`
- Windows (MinGW g++): thêm cờ khi biên dịch: `g++ -Wl,-stack,268435456 main.cpp -o main`
- Python: `import sys; sys.setrecursionlimit(10**6)`

Sai lầm 3: Nhầm lẫn đánh số 0-indexed và 1-indexed

Đề bài thường đánh số đỉnh từ 1 đến n , nhưng lưới thường đánh số từ 0 đến $n - 1$. Nếu dùng sai, sẽ bị lỗi truy cập ngoài mảng.

C++

```

// Nếu đỉnh đánh số 1..n -> khai báo mảng kích thước n+1
vector<vector<int>> adj(n + 1);
bool visited[n + 1];

// Nếu lưới đánh số 0..n-1 -> chú ý kiểm tra biên
if (nx >= 0 && nx < n && ny >= 0 && ny < m) // ĐÚNG
if (nx >= 1 && nx <= n && ny >= 1 && ny <= m) // Nếu đánh số từ 1

```

Python

```

# Nếu đỉnh đánh số 1..n -> thường tạo mảng kích thước n+1
adj = [[] for _ in range(n + 1)]
visited = [False] * (n + 1)

# Nếu lưới đánh số 0..n-1 -> chú ý kiểm tra biên

```

```

if 0 <= nx < n and 0 <= ny < m: # ĐÚNG
    pass
# Nếu bạn cố dùng kiểu 1..n cho lưới thì phải cực kỳ nhất quán,
# nhưng thông thường grid trong Python nên dùng 0-indexed

```

Sai lầm 4: Đồ thị vô hướng nhưng chỉ thêm cạnh một chiều C++

```

// SAI - đồ thị vô hướng cần cạnh hai chiều!
adj[u].push_back(v);

// ĐÚNG
adj[u].push_back(v);
adj[v].push_back(u);

```

Python

```

# SAI - đồ thị vô hướng cần cạnh hai chiều!
adj[u].append(v)

# ĐÚNG
adj[u].append(v)
adj[v].append(u)

```

Sai lầm 5: BFS nhưng đánh dấu visited SAU khi lấy ra khỏi queue C++

```

// SAI - cùng đỉnh có thể được thêm vào queue nhiều lần!
while (!q.empty()) {
    int u = q.front(); q.pop();
    visited[u] = true; // <- quá muộn!
    for (int v : adj[u])
        if (!visited[v])
            q.push(v);
}

// ĐÚNG - đánh dấu NGAY KHI thêm vào queue
while (!q.empty()) {
    int u = q.front(); q.pop();
    for (int v : adj[u]) {
        if (dist[v] == -1) { // Dùng dist == -1 thay cho visited
            dist[v] = dist[u] + 1;
            q.push(v);
        }
    }
}

```

Python

```
# SAI - cùng đỉnh có thể được thêm vào queue nhiều lần!
while q:
    u = q.popleft()
    visited[u] = True # <- quá muộn!
    for v in adj[u]:
        if not visited[v]:
            q.append(v)

# ĐÚNG - đánh dấu NGAY KHI thêm vào queue
while q:
    u = q.popleft()
    for v in adj[u]:
        if dist[v] == -1: # Dùng dist == -1 thay cho visited
            dist[v] = dist[u] + 1
            q.append(v)
```

10. Mẹo luyện tập

- Bắt đầu từ bài DFS/BFS thuần: đếm thành phần liên thông, flood fill.
- Luôn vẽ đồ thị ra giấy khi đọc đề — nhiều bài trông phức tạp nhưng vẽ ra sẽ rõ.
- Khi bài cho lưới: nghĩ ngay đến BFS/DFS trên grid với 4 hướng dx/dy .
- Khi bài yêu cầu đường đi ngắn nhất (không trọng số): BFS.
- Khi bài yêu cầu kiểm tra liên thông, đếm vùng: DFS.
- Khi bài có thứ tự phụ thuộc: topological sort.
- Khi bài yêu cầu chia 2 nhóm: kiểm tra bipartite.

Đồ thị là chủ đề rất rộng — BFS và DFS chỉ là bước khởi đầu. Nhưng chỉ với hai thuật toán này, các bạn đã có thể giải được rất nhiều bài!